PROJECT REPORT

Project 2: Formal Criteria-Based Test Design & Automation

Premium Discount Eligibility System

Input Space Partitioning | Pair-Wise Coverage | JUnit 5

1. System Overview

The system under test is the DiscountCalculator class, which calculates a discount percentage based on a customer profile. The calculation applies a fixed base discount and conditionally adds bonuses based on customer type, newsletter subscription status, and purchase loyalty. The total discount is capped at 15%.

1.1 Functional Rules

- **Rule 1 –** Base Discount: All customers receive a 5% base discount.
- **Rule 2 –** Subscription Bonus: Customers subscribed to the newsletter receive an additional 2%.
- **Rule 3 –** Customer Type Bonus: NEW = +0%, REGULAR = +3%, PREMIUM = +5%.
- **Rule 4 –** Loyalty Bonus: 10 or more orders in the last year adds 5%.
- **Rule 5 –** Infeasibility Constraint: A NEW customer cannot have 10 or more orders (throws IllegalArgumentException).
- **Rule 6 –** Maximum Cap: The total discount is capped at 15%.

2. Input Space Partitioning (ISP)

Model-Driven Test Design (MDTD) begins by identifying the characteristics of the input domain. Each characteristic is partitioned into disjoint and complete blocks that cover all possible values with no overlaps.

2.1 Identified Characteristics

- C1 – customerType: The type of customer (NEW, REGULAR, PREMIUM).
- C2 – totalOrdersInLastYear: The number of orders placed in the past year (integer ≥ 0).
- C3 – isSubscribedToNewsletter: Whether the customer is subscribed (boolean).

2.2 Characteristic Blocks (Disjoint & Complete)

Characteristic	Block ID	Block Description	Representative Value
customerType	B1a	NEW customer	NEW

Characteristic	Block ID	Block Description	Representative Value
	B1b	REGULAR customer	REGULAR
	B1c	PREMIUM customer	PREMIUM
totalOrdersInLastYear	B2a	Less than 10 orders	5
	B2b	10 or more orders	12
isSubscribedToNewsletter	B3a	Subscribed to newsletter	true
	B3b	Not subscribed to newsletter	false

The blocks above are disjoint (no value belongs to two blocks for the same characteristic) and complete (every possible valid input value falls into exactly one block per characteristic). The binary nature of C3 and the threshold on C2 ensure full coverage of the input domain.

3. Infeasible Combination Justification

One combination across the characteristics is logically infeasible:

Infeasible Pair: customerType = NEW AND **totalOrdersInLastYear ≥ 10 (Block B2b)**

Justification: A NEW customer is, by definition, someone who has just joined or is making their first purchases. It is a business-domain impossibility for such a customer to have accumulated 10 or more orders in the last year. This constraint is not a test design choice but a real-world domain rule embedded in the system specification.

Implementation: The DiscountCalculator.calculateDiscount() method checks this condition first and throws an IllegalArgumentException with a descriptive message. In the test suite, this is validated using assertThrows(IllegalArgumentException.class, ...) inside a @ParameterizedTest covering multiple values (10, 15, 100 orders).

4. Derived Pair-Wise Coverage (PWC) Test Cases

Pair-Wise Coverage (PWC) requires that every pair of values from any two characteristics appears together in at least one test case. With three characteristics (C1: 3 values, C2: 2 values, C3: 2 values), the pairs to be covered are: (C1 × C2), (C1 × C3), and (C2 × C3).

4.1 PWC Test Case Matrix

#	customerType	Orders	Subscribed	Expected	Status	Calculation
1	NEW	5	true	7%	Feasible	5+2+0 = 7
2	NEW	5	false	5%	Feasible	5+0+0 = 5

#	customerType	Orders	Subscribed	Expected	Status	Calculation
3	REGULAR	5	true	10%	Feasible	5+2+3 = 10
4	REGULAR	12	false	13%	Feasible	5+0+3+5 = 13
5	PREMIUM	5	false	10%	Feasible	5+0+5 = 10
6	PREMIUM	12	true	15%	Feasible	5+2+5+5=17 -> cap 15
7	REGULAR	12	true	15%	Feasible	5+2+3+5 = 15
8	PREMIUM	12	false	15%	Feasible	5+0+5+5 = 15
IF	NEW	>=10	any	N/A	**INFEASIBLE**	IllegalArgumentException

4.2 Pair Coverage Verification

- C1 × C2 – All 3 customer types paired with <10 and ≥10 orders: covered in rows 1–8 (NEW×<10: rows 1,2; REGULAR×<10: row 3; REGULAR×≥10: rows 4,7; PREMIUM×<10: row 5; PREMIUM×≥10: rows 6,8). NEW×≥10 is infeasible (row IF).
- C1 × C3 – NEW×true: row 1, NEW×false: row 2, REGULAR×true: rows 3,7, REGULAR×false: row 4, PREMIUM×true: row 6, PREMIUM×false: rows 5,8.
- C2 × C3 – <10×true: row 1,3, <10×false: rows 2,5, ≥10×true: rows 6,7, ≥10×false: rows 4,8.

5. Limitations of Pair-Wise Testing

Pair-Wise Coverage (PWC) is a powerful combinatorial technique that significantly reduces the number of required test cases while maintaining reasonable fault-detection capability. However, it does not guarantee a bug-free system for several important reasons:

5.1 Missing 3-Way (and Higher-Order) Interaction Faults

PWC guarantees that every pair of parameter values is tested together at least once, but it does not cover all three-way or higher-order interactions. In the DiscountCalculator, a fault that only manifests when customerType = PREMIUM AND totalOrdersInLastYear >= 10 AND isSubscribedToNewsletter = true simultaneously could go undetected if no single test case exercises that exact triple. For instance, if there were a subtle rounding bug in the cap logic triggered only by that three-way interaction, PWC tests would not find it because none of the individual pairs would reveal it.

5.2 No Coverage of Edge Values Within Blocks

ISP blocks are treated as monolithic units; PWC selects one representative value per block. The boundary between Block B2a (<10) and Block B2b (>=10) is particularly critical: the value 9 (last value excluded from loyalty) and 10 (first value included) are never both tested against all other parameter combinations. A classic off-by-one error in the loyalty threshold condition (e.g., > 10 instead of >= 10) would not be detected by pair-wise testing alone unless boundary-value analysis is layered on top.

5.3 Infeasible Combinations Reduce Coverage

The infeasible pair (NEW, >=10) means that PWC cannot achieve full combinatorial coverage even within two-way interactions. This structural gap cannot be remedied by adding more test cases—it is a permanent hole in the pair-wise matrix. Any fault that could only be exposed via that pair remains untestable, and the tester must rely solely on the exception guard to enforce the constraint.

5.4 No Behavioral or Sequence Testing

PWC focuses exclusively on input combinations; it says nothing about the order in which methods are called, how the object behaves across multiple invocations, or whether the system handles concurrent access correctly. A stateful version of DiscountCalculator that accumulated discounts across calls would require sequence-based testing (e.g., finite-state machine testing) that PWC cannot provide.

5.5 Conclusion

Pair-Wise Coverage is a valuable technique for reducing a combinatorial explosion of test cases while still exposing a large proportion of pairwise interaction faults. However, it must be complemented with boundary-value analysis, 3-way (or higher) combinatorial testing where risk warrants it, and domain-specific constraint validation to approach comprehensive fault detection.

6. Team Contribution Matrix

All members collaborated on design decisions. Individual primary responsibilities are documented below and are reflected in the GitHub commit history, where each member committed exclusively from their own personal account.

Team Member	Contribution	Report Sections	Rubric Criteria
Yahya Abu Shawish ID: 2232480	Implemented DiscountCalculator.java: all discount rules (base, subscription, customer type, loyalty bonus), cap logic, and infeasibility guard (IllegalArgumentException)	Sections 1, 2, 3	A (MDTD & ISP Design)
Mustafa Nassar ID: 2330208	Implemented DiscountCalculatorTest.java: JUnit 5 @ParameterizedTest with @CsvSource for all PWC pairs, assertThrows for infeasible combination, limitations analysis, and report writing	Sections 4, 5, 6	B, C, D (Parameterized Testing, Limitations, Teamwork)